Final Group Work Report: RISC-V Pipelined Processor on PYNQ-Z2

Oscar Edgardo Navarro Banderas
Matr. Nr. 03782200

Beyrem Hadj Fredj
Matr. Nr. 03798143

Leon Adomaitis
Matr. Nr. 03789537

20.02.2026

Contents

1 Executive Summary

This report documents the final version of our pipelined RISC-V core on the PYNQ-Z2 board. The practical shift compared to earlier labs is that verification is no longer done only in a Verilog testbench: the PS side (ARM + Python) drives and checks the PL implementation directly.

In the final flow, Python loads instructions and data into BRAM, releases reset through AXI GPIO, waits for the DONE flag, and compares memory output against a software golden reference. We chose split instruction/data BRAMs to keep the integration manageable and to isolate bugs faster during bring-up.

The design runs end-to-end on board. The report includes the measured utilization and timing values from the final implementation run.

2 Design Overview

2.1 Project Goal and Verification Concept

The goal is to validate the pipelined RISC-V core under real FPGA timing, not only behavioral simulation. In our setup, the PS prepares memory, starts/stops execution, and checks results, while the PL executes the sorting workload and writes completion status. This split made debugging much clearer than a pure RTL-only loop.

2.2 System Architecture

The implemented block design keeps the usual Zynq PS + AXI backbone from the lab setup, then adds the project-specific pieces: AXI BRAM controllers for instruction/data access, AXI GPIO reset control, dual-port BRAM generators, and our packaged pipelined RISC-V IP.

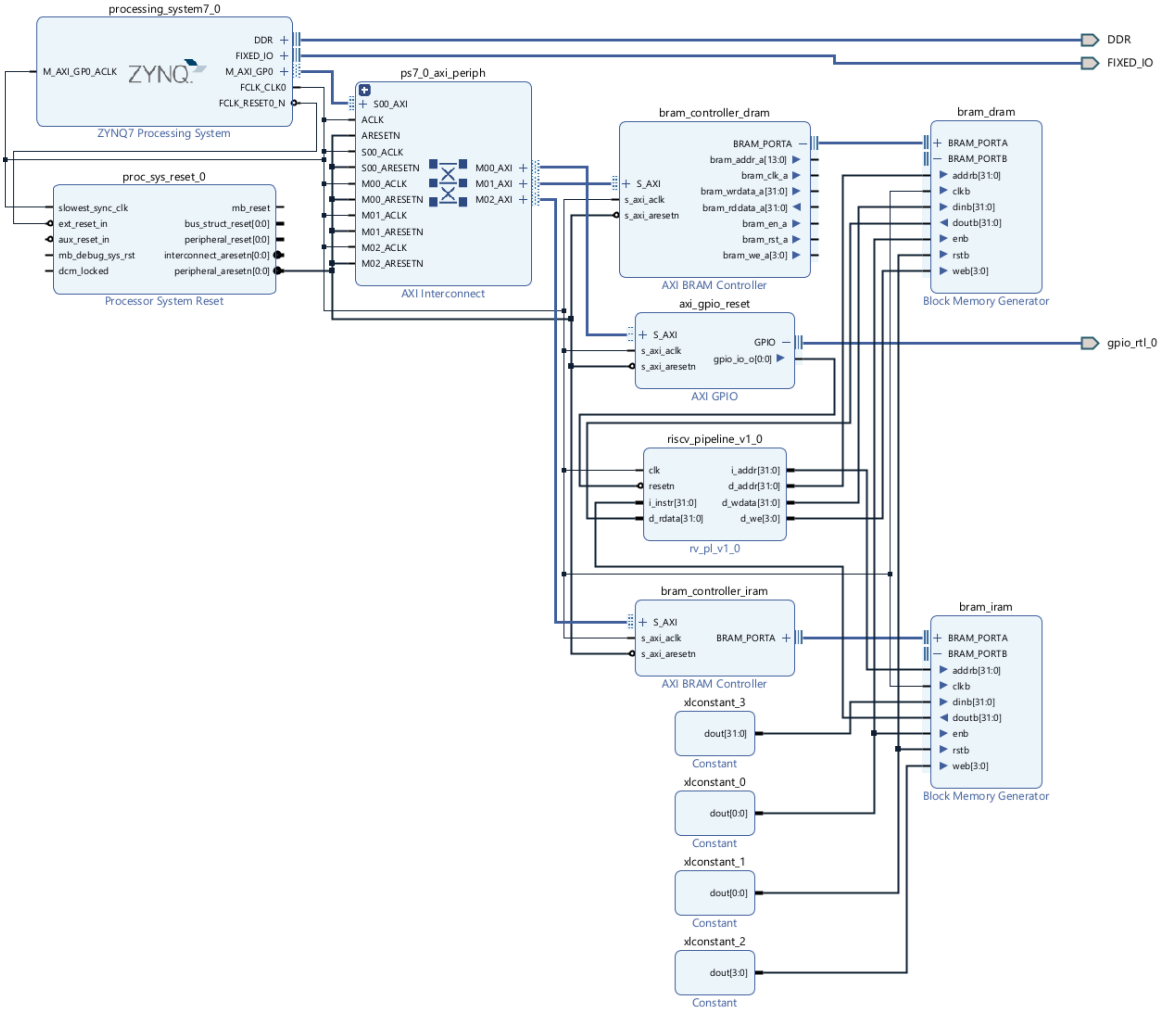

Figure 1: Vivado block design used for PS-PL integration and verification flow.

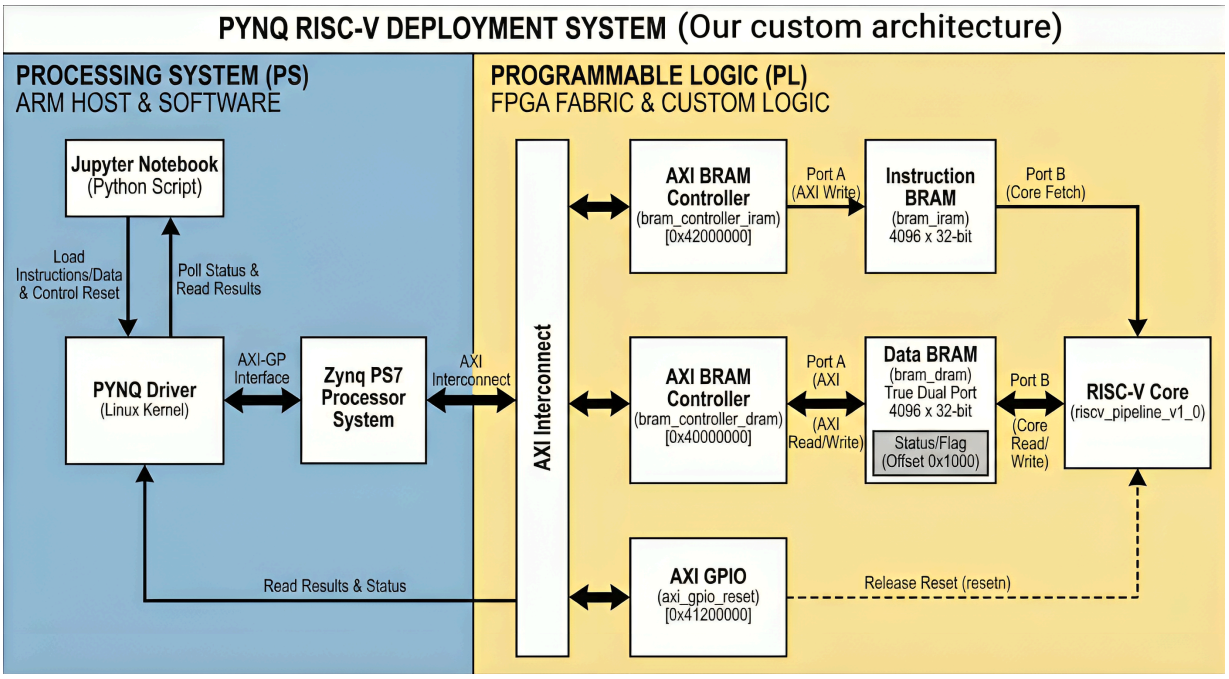

Figure 2: Custom PS-PL deployment architecture used during project integration and demo preparation.

During runtime, the Python notebook accesses three memory-mapped regions:

Block	Base Address	Role in verification
`bram_controller_dram`	`0x40000000`	Data memory writes/reads and DONE polling
`bram_controller_iram`	`0x42000000`	Instruction injection and readback verification
`axi_gpio_reset`	`0x41200000`	Core halt/run control

2.3 Memory Layout Decision

The assignment allows both unified and split memory layouts. We chose split instruction/data BRAM because it reduced contention problems and cut debugging time. A unified memory is still a valid option, but it needs stricter arbitration and more careful stall validation.

2.4 Integration Deltas from the Lab Baseline

We started from the Lab 12 style PS-PL architecture and kept the proven communication structure, then introduced only the changes needed for the final project. AXI SmartConnect was replaced by AXI Interconnect in the final automated flow, a second BRAM controller was added to split IRAM and DRAM, and the `rv_pl` wrapper was packaged as a reusable IP block.
This incremental approach kept integration stable late in the project and made failures easier to localize.

3 Module Description and Implementation

3.1 Pipeline Structure

The processor uses five classic stages (`IF`, `ID`, `EX`, `MEM`, `WB`) and dedicated pipeline registers (`fd_register`, `de_register`, `em_register`, `mw_register`).

Stage	Main function	Representative modules
IF	PC update and instruction fetch	`MyPipelinedProcessor.v`, `MyProgramCounter.v`, `fd_register.v`
ID	Decode, immediate generation, register read	`MyController.v`, `MyControlUnit.v`, `MyInstructionDecoder.v`, `MyRegisterFile.v`, `de_register.v`
EX	ALU operations, branch target, operand forwarding	`MyALUDecoder.v`, `MyAlu.v`, `3mux.v`, `de_register.v`, `em_register.v`
MEM	Data memory access and write-enable control	`MyPipelinedProcessor.v`, `em_register.v`, `mw_register.v`
WB	Result selection and register write-back	`MyPipelinedProcessor.v`, `mw_register.v`

3.2 Architectural Modifications for FPGA Deployment

Moving the lab core to FPGA required a few targeted architectural changes:

1. Hazard unit extension for synchronous memory behavior: load and store interactions were re-checked under one-cycle BRAM latency assumptions, with explicit stall/forward coordination.
2. Top-level BRAM interface exposure: `rv_pl` exports instruction and data memory ports directly (`i_addr/i_instr`, `d_addr/d_wdata/d_we/d_rdata`) for clean block-design integration.
3. Instruction buffering during stalls: an explicit buffer in fetch/decode path prevents instruction corruption across stall-release transitions.

These modifications were necessary for stable board behavior; functional simulation alone did not expose all of these effects. We also kept sequential updates in clocked `always @(posedge clk)` processes so BRAM-latency assumptions stayed explicit during integration.

3.3 Decode and ALU Control Path

Control logic is split into two levels. `MyControlUnit.v` decides high-level instruction class behavior (`reg_write`, `mem_write`, `alu_src`, `result_src`, `branch`, `jump`, `alu_op`), while `MyALUDecoder.v` maps opcode/function bits to concrete ALU operations. In practice, this made debugging easier because decode-intent errors and ALU-mapping errors could be separated quickly.

3.4 Hazard and BRAM Timing Handling

The main board-specific challenge was synchronous BRAM timing. Because read data arrives one phase later, fetch, decode, and memory control had to stay aligned across cycles. In practice, this meant combining regular RAW forwarding and load-use stalls with BRAM-aware stall gating, fetch alignment, instruction buffering around stall windows, and one extra flush guard near branch redirects.

3.5 Program Behavior and DONE Protocol

The sorting workload processes 32 signed integers in place in DRAM (`0x00` to `0x7C`). Completion is signaled by storing `0xDEADBEAF` at byte address `0x100`.

During development we used two program styles: an unrolled adjacent compare-swap generator (from the notebook side for board verification), and a compact loop-based bubble sort program (`sort_32_bubble.hex`) for simulation/debug runs.

The unrolled network contains `2,980` instructions, which fits in the `4,096`-word IRAM space.

Representative compare-swap block:

```
lw   x7, off(x0)
lw   x8, off+4(x0)
slt  x9, x8, x7
beq  x9, x0, +12
sw   x8, off(x0)
sw   x7, off+4(x0)
```

The DONE constant is built with `lui x12, 0xDEADC`, then `addi x12, x12, -337`, and finally written with `sw x12, 256(x0)`.

This constant build pattern comes from ISA limits and is also the handshake point used by the host script.

4 Testing and Debugging

4.1 Pre-Hardware Simulation

The project includes a dedicated testbench (`tb_rv_pl.v`) with synchronous BRAM modeling (address registered on clock edge, data observed on the following cycle), which is closer to real FPGA behavior than idealized combinational memories.

The test sequence is intentionally progressive:

1. basic store operations,
2. store-then-load behavior,
3. load-use hazard case,
4. consecutive loads plus dependent arithmetic,
5. branch-taken behavior,
6. mini bubble sort (4 elements),
7. full 32-element sort with done-flag check.

This structure let us isolate control and memory-timing issues before running the full workload.

In team simulation notes, the full 32-element test completes in roughly 9,609 cycles, which is in line with an in-order bubble-sort workload with frequent memory traffic.

Tests 1-5 isolate individual pipeline behaviors. Test 6 keeps waveforms readable while validating end-to-end sorting behavior on a smaller array. Test 7 runs the full 32-element workload.

4.2 Cross-Simulator Discrepancy Notes

One development lesson was simulator mismatch. During debugging, `iverilog` and Vivado `xsim` did not always behave the same way, and unknown (`x`) propagation appeared in `xsim` runs. The main cause was initialization assumptions: the register file and selected control/data registers needed explicit reset/initialization discipline, and BRAM-address staging registers in the testbench needed deterministic startup values.

Treating initialization explicitly improves portability and reduces "works here, fails there" ambiguity across simulators.

Typical mitigation pattern:

```
initial begin
    i_addr_reg = 32'b0;
    d_addr_reg = 32'b0;
end
```

4.3 Hardware-in-the-Loop Verification Flow

The Jupyter notebook (`verify_submission.ipynb`) follows a deterministic run sequence:

1. load overlay,
2. map MMIO peripherals,
3. execute memory and GPIO sanity checks,
4. inject program and input data,
5. release reset,
6. poll status flag,
7. compare output with Python `sorted()` golden result.

The notebook acts as the board-level harness: completion is detected by status polling, and output ordering is checked against a software golden result.

Before running the sorter, the notebook performs sanity checks on IRAM, DRAM, and GPIO. We used write/readback patterns such as `0xDEADBEEF`, `0xCAFEBABE`, and address-tagged values (`0xAA000000 | offset`) at offsets `0x0`, `0x4`, `0x10`, `0x7C`, and `0x100`. This separates integration faults (addressing, MMIO mapping, reset control) from CPU-logic bugs.

4.4 On-Board Execution Profile (Captured Run)

Metric	Observed value
Program length	2,980 instructions (generated unrolled adjacent compare-swap network)
Input size	32 signed integers
DONE flag address	`0x100` (byte offset in DRAM)
DONE flag value	`0xDEADBEAF`
Observed completion behavior	DONE flag polled until set by the core
Result check	Output compared to Python `sorted()` golden reference

This profile ties memory map, control flow, and verification outcome into one snapshot.

4.5 Debugging Experience: What Failed and Why

The hardest bugs were not arithmetic bugs in the sorter itself. Most failures came from control/timing alignment across pipeline stages when BRAM latency was involved.

Observed symptom	Technical reason	Fix applied
Unstable or inconsistent board output	Address/data timing mismatch around BRAM reads	Introduced BRAM-aware stalling and fetch alignment logic
Wrong instruction after stall release	Instruction stream advanced while valid fetch data lagged	Added instruction buffering for stall transitions

Control transfer irregularities	One-cycle stale fetch behavior after branch redirection	Added extra flush handling near branch redirect

A second lesson was tool behavior: RTL that looks fine in one flow can expose stricter elaboration behavior in another simulator. Cross-tool checks should start earlier.

4.6 On-Board Failure Case and Fix

A representative board issue was instruction-sequencing corruption around stall windows: after a stall, the next decoded instruction could be misaligned with the intended fetch stream. The practical fix was to latch the instruction when entering stall and replay it safely until normal flow resumed.

```
always @(posedge clk) begin
    if (!resetn || FD_flush_final)
        instr_buf_valid <= 1'b0;
    else if (!F_stall)
        instr_buf_valid <= 1'b0;
    else if (F_stall && !instr_buf_valid) begin
        instr_buf       <= i_instr;
        instr_buf_valid <= 1'b1;
    end
end
wire [31:0] fd_instr_in = instr_buf_valid ? instr_buf : i_instr;
```

This fix is a good example of why board timing behavior must be treated as a design constraint, not an afterthought.

4.7 Bring-Up Lessons

Beyond RTL issues, several integration mistakes repeatedly blocked progress during board bring-up. We initially treated BRAM controllers as regular IP entries, while PYNQ exposes them through `mem_dict`, which caused false “missing memory” diagnostics. Notebook naming was also inconsistent (`bram_controller_irom` vs `bram_controller_iram`), so mapping broke even with correct hardware. During rapid iteration, stale wrappers or old bitstreams were occasionally tested by mistake, producing contradictory results between simulation and board runs. One early generated sort image exceeded BRAM capacity (`17,304` lines for a `4,096`-word BRAM), so that version could not execute correctly on hardware. We also saw misleading failures when host polling/read checks were done before valid data became visible.

These were PS/PL integration issues, not algorithmic mistakes in bubble sort. They explain why “simulation passes” did not immediately mean “board passes.”

5 FPGA Implementation Analysis

This section summarizes the post-implementation Vivado data and the main bottlenecks we observed.

5.1 Hierarchical Resource Utilization

Module	LUTs	FFs	LUT-Logic	LUT-Mem	BRAM
`riscv_pynq_wrapper`	2528	2460	2406	122	8
`riscv_pipeline_v1_0` (total)	976	596	932	44	0
`Branch_Adder`	0	0	0	0	0
`HazardUnit (HU)`	0	1	0	0	0
`ProgramCounter (PC)`	0	32	0	0	0
`PC_Adder`	0	0	0	0	0
`PC_R_Adder`	1	0	1	0	0
`PLR1 (FD_register)`	136	94	136	0	0

PLR2 (DE_Register)	438	183	438	0	0
PLR3 (EM_Register)	10	116	10	0	0
PLR4 (MW_Register)	337	104	337	0	0
ALU	16	0	16	0	0
RegisterFile (RF)	44	0	0	44	0

The dominant LUT consumer inside the core is `DE_Register`. That is expected: the decode-to-execute boundary carries operand buses, register indices for hazard decisions, and multiple control signals. The logic cost is concentrated at this stage boundary rather than in one arithmetic block.

5.2 Critical Path Analysis

From the implementation notes, the reported worst path starts near a memory-stage register index signal and ends at an execute-stage immediate register (`.../M_rf_a3_reg[4]/C -> .../PLR2/E_ext_reg[21]/R`).
This suggests the timing limit is not mainly BRAM setup time and not one ALU operator. The bottleneck is more consistent with combined control propagation and selection logic around execute-stage forwarding paths.

Name	Slack ^1	Levels	High Fanout	From	To
Path 1	-2.399	14	183	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR3/M_rf_a3_reg[4]/C	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR2/E_ext_reg[21]/R
Path 2	-2.399	14	183	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR3/M_rf_a3_reg[4]/C	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR2/E_rf_rd1_reg[12]/R
Path 3	-2.399	14	183	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR3/M_rf_a3_reg[4]/C	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR2/E_rf_rd2_reg[6]/R
Path 4	-2.399	14	183	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR3/M_rf_a3_reg[4]/C	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR2/E_rs1_reg[3]/R

Figure 3: Vivado timing report excerpt for the top violating paths.

In the captured timing table, multiple top paths share the same pattern: `Slack = -2.399 ns`, `Levels = 14`, and `High Fanout = 183`, with a common source around `PLR3/M_rf_a3_reg[4]/C` and endpoints in `PLR2` registers (`E_ext`, `E_rf_rd1`, `E_rf_rd2`, `E_rs1`). This points to control/metadata fanout around the decode-execute boundary as a major timing contributor.
Practically, delay is spread across forwarding mux selection, branch-target/operand interaction, and execute-stage metadata propagation.
Representative execute-stage structure:

```
ThreeMux MuxA (
  .in0(E_rf_rd1), .in1(W_result), .in2(M_alu_o),
  .sel(ForwardAE), .out(E_src_a_forwarded)
);

ThreeMux MuxB (
  .in0(E_rf_rd2), .in1(W_result), .in2(M_alu_o),
  .sel(ForwardBE), .out(E_src_b_forwarded)
);

Adder Branch_Adder (
  .a(E_pc), .b(E_ext), .sum(E_target_pc)
);

assign E_alu_src_b = (E_sel_alu_src_b) ? E_ext : E_src_b_forwarded;
```

5.3 Timing Performance and Fmax

Using the reported values:

- `WNS = -2.399 ns`

- `T_constraint = 10.000 ns` (100 MHz target)

A negative setup slack means the design misses the target period and requires a longer effective clock period:

- `T_clk,min = T_constraint + |WNS| = 12.399 ns`
- `Fmax = 1 / T_clk,min = 80.65 MHz` (approx.)

These values explain why timing closure at 100 MHz was not achieved in the cited run, even though functional verification passed.

6 Technical Discussion

6.1 Tradeoff: Split Memories vs Unified Memory

We chose split memories because they gave us a clearer debug path and fewer structural hazards to handle under deadline pressure. A unified BRAM layout can reduce block count, but it moves more complexity into fetch/load-store arbitration.

6.2 Reset Strategy on Board

Reset handling on board is critical for repeatability. We used AXI GPIO because it is scriptable and deterministic in automated tests. A physical button is fine for demos, but needs debounce logic to avoid jitter-related false triggers.

6.3 What We Would Improve Next

If we continue after the final submission, the next steps would be:

- automate Vivado report extraction into report tables,
- add cycle/performance counters for deeper profiling,
- optionally add a unified-memory variant for direct architectural comparison.

The optional single-cycle bonus track was not part of this submission scope.

7 Conclusion

We met the project objective: the pipelined RISC-V core was integrated on PYNQ-Z2 and validated with a repeatable PS-driven hardware-in-the-loop flow. Sorting output is written in place, and completion is detected reliably through the DONE flag protocol.

The main technical takeaway is that FPGA correctness depends as much on timing-aware control as on instruction-level logic. The design only stabilized when BRAM timing, hazard policy, and reset handling were treated as one connected problem.

If we continued, the first optimization target would be timing closure around execute/control fanout paths, followed by automated extraction of implementation metrics into the report.

8 Contributions

The workload was split fairly based on available time and task type.

Team Member	**Primary Contributions**	**Weight**
Leon Adomaitis	Code documentation, report writing, and slides preparation	Medium
Beyrem Hadj Fredj	Hardware programming, debugging, and code documentation	High
Oscar Banderas	Code documentation, Vivado block-design preparation, and testing/verification	High

Shared contribution: all members participated in debugging decisions and final integration discussions.

9 Repository and Submission Artifacts

Project repository used for submission:

- `https://github.com/Oscar27NX/hdl-ws25-26-final-project.git` (branch: `RV_PYNQ`)

Main artifact groups available in the repository:

- RTL source and pipeline modules,
- block-design wrapper and integration files,
- simulation testbench and hex programs,
- bitstream and hardware handoff (`.bit`, `.hwh`),
- verification notebook and software-side test material.

Representative paths:

- `src/RTL/riscv_core_ip_source/`,
- `src/RTL/block_design/riscv_pynq_wrapper.v`,
- `src/RTL/simulation/tb_rv_pl.v`,
- `src/software/test_sort_docu.s`, `src/software/test_sort.hex`,
- `src/verification_script/verify_submission.ipynb`,
- `src/bit_and_hwh/riscv_pynq_lfg.bit`, `src/bit_and_hwh/riscv_pynq_lfg.hwh`.

10 Citations and Disclosure

10.1 Citations

- Final project requirements PDF (course handout, January 27, 2026).
- PYNQ documentation for overlay/MMIO workflow: `https://pynq.readthedocs.io/`.
- Repository artifacts used as primary evidence: RTL sources, simulation testbench, verification notebook, block-design wrapper, and hardware handoff file.
- TUM Typst theme reference: `https://github.com/lufixSch/tum-templates-typst`.

10.2 AIGC Disclosure

AI assistance was limited to language-level style refinement. Technical content, implementation claims, and verification statements were reviewed against repository artifacts by the team.